

01 - Lista de Exercícios: Java e Orientação a Objetos

Parte 1: Questões Objetivas (10 Questões)

1. Em relação às Exceções Customizadas em Java, para criar uma exceção de negócio do tipo *não checada* (unchecked exception), a classe personalizada deve herdar diretamente de:
 - a) Throwable
 - b) Exception
 - c) RuntimeException
 - d) Error
2. Qual interface da API JDBC é recomendada para executar consultas SQL parametrizadas, prevenindo ataques de *SQL Injection* e otimizando a performance do banco de dados?
 - a) DriverManager
 - b) Statement
 - c) PreparedStatement
 - d) ResultSet
3. Para iniciar uma nova linha de execução paralela na JVM a partir de um objeto da classe Thread, qual método deve ser obrigatoriamente chamado pelo desenvolvedor?
 - a) t.run()
 - b) t.start()
 - c) t.execute()
 - d) t.init()
4. Sobre as palavras reservadas extends e implements, assinale a alternativa correta:
 - a) Uma classe Java pode estender (extends) múltiplas superclasses simultaneamente.
 - b) Uma classe Java pode herdar de uma superclasse com extends e implementar múltiplas interfaces com implements.
 - c) A palavra implements é utilizada para estabelecer relação de herança entre duas classes concretas.
 - d) A palavra extends é utilizada exclusivamente para implementar métodos abstratos declarados em interfaces.
5. Qual é a assinatura exata do método de ponto de entrada (*entry point*) exigido pela JVM para iniciar a execução de uma aplicação Java tradicional?
 - a) public void main(String[] args)
 - b) public static int main(String args)
 - c) public static void main(String[] args)
 - d) private static void main(String[] args)

6. Se um método em Java for declarado com o modificador de acesso `protected`, ele poderá ser acessado por:
- a) Apenas métodos contidos na própria classe.
 - b) Qualquer classe do mesmo pacote e por subclasses em qualquer pacote.
 - c) Exclusivamente por subclasses, independentemente do pacote.
 - d) Qualquer classe presente em qualquer pacote do projeto.
7. Quando um desenvolvedor declara um método sem especificar nenhum modificador de acesso (`public`, `protected` ou `private`), qual é o escopo padrão (*default*) aplicado?
- a) O método fica acessível a todo o projeto (*public*).
 - b) O método só pode ser acessado na própria classe (*private*).
 - c) O método fica acessível apenas para classes do mesmo pacote (*package-private*).
 - d) O método torna-se automaticamente estático (*static*).
8. No contexto do processamento JDBC, qual método da interface `ResultSet` é utilizado para mover o cursor para a próxima linha da tabela de resultados retornada pelo SGBD?
- a) `rs.getLine()`
 - b) `rs.next()`
 - c) `rs.forward()`
 - d) `rs.move()`
9. Sobre o conceito de encapsulamento em Java, qual é a boa prática recomendada para expor funcionalidades de uma classe com segurança?
- a) Manter os atributos como `public` e os métodos como `private`.
 - b) Manter o estado (atributos) como `private` e disponibilizar métodos de acesso públicos (`public`) ou protegidos (`protected`).
 - c) Declarar todos os métodos da classe como `protected` para que fiquem acessíveis a qualquer outra classe do projeto.
 - d) Evitar o uso de métodos de validação internos em métodos privados.
10. Qual é a principal diferença entre concorrência e paralelismo no contexto de processamento de Threads?
- a) Concorrência executa tarefas em múltiplos núcleos de CPU físicos; paralelismo simula tempo compartilhado em um único núcleo.
 - b) Concorrência é a gestão de múltiplas tarefas que podem progredir alternadamente; paralelismo é a execução simultânea real de tarefas em múltiplos núcleos.
 - c) Concorrência refere-se apenas a chamadas HTTP; paralelismo refere-se apenas a consultas de banco de dados.
 - d) Não há diferença prática entre os dois termos na JVM.

Parte 2: Questões Subjetivas (5 Questões)

1. (Exceções Customizadas): Explique por que é considerado uma boa prática de arquitetura criar exceções de negócio personalizadas (ex: `VeiculoIndisponivelException`) em vez de utilizar exceções genéricas como `RuntimeException` ou `IllegalArgumentException`.
2. (API JDBC e Segurança): Considere um cenário onde é necessário buscar um veículo na tabela `veiculos` informando a placa via parâmetro. Escreva um trecho de código Java que utilize as interfaces `Connection`, `PreparedStatement` e `ResultSet` para executar a consulta de forma segura usando o bloco `try-with-resources`.
3. (Threads e Concorrência): O que acontece quando o método `.run()` é chamado diretamente em uma instância de `Thread` em vez do método `.start()`? Explique o impacto no fluxo de execução do programa e na criação de novas linhas de processamento na JVM.
4. (POO: Herança e Interfaces): Considere o ecossistema de gestão de frotas. Escreva a definição em Java para:
 - a. Uma interface chamada `Rastreavel` que declara o método `void atualizarPosicao(double lat, double lng)`;
 - b. Uma classe chamada `Caminhao` que herda de uma superclasse `Veiculo` e implementa a interface `Rastreavel`.
5. (Encapsulamento e Visibilidade): Analise o trecho de código abaixo:

```
01. public class ProcessadorPagamento {
02.     public void executarTransacao(double valor) {
03.         if (validarFraude(valor)) {
04.             System.out.println("Transação aprovada!");
05.         }
06.     }
07.
08.     private boolean validarFraude(double valor) {
09.         return valor < 5000.0;
10.     }
11. }
```

Explique o motivo de o método `validarFraude` ter sido declarado como `private`. O que aconteceria com a integridade da aplicação se ele fosse alterado para `public`?

Parte 3: Questões Práticas e Desafios (3 Seções)

Seção 1: Exercícios de Fixação (Conceitos Básicos)

Exercício 1 [Expansão do Modelo de Herança e Polimorfismo]: A frota da empresa agora precisa gerenciar **Caminhões**.

1. Crie uma nova subclasse chamada **Caminhao** que herde de **Veiculo**.
2. Adicione o atributo privado **capacidadeCargaToneladas** do tipo **double** e seus métodos acessores.
3. Sobrescreva o método **exibirFichaTecnica()** para que a saída do caminhão inclua a capacidade de carga formatada (ex: **[CAMINHAO] Volvo FH - Ano: 2022 | Carga: 25.5t**).
4. Atualize o método **ConexaoBanco.salvarVeiculo()** e a rotina de busca **ConexaoBanco.listarVeiculos()** para suportar a persistência e leitura de objetos do tipo **Caminhao**.

Exercício 2 [Tratamento e Validação com Exceções Customizadas]: Atualmente, o sistema lança **DadosVeiculoInvalidosException** apenas quando o ano do veículo é inválido.

1. Altere o método **salvarVeiculo()** da classe **ConexaoBanco** para validar as seguintes regras de negócio antes de salvar no banco de dados:
 - a. **Marca e Modelo** não podem ser vazios nem conter apenas espaços em branco (**String.isBlank()**).
 - b. Para a subclasse **Carro**, a **quantidadePortas** deve ser no mínimo 2 e no máximo 6.
 - c. Para a subclasse **Moto**, as **cilindradas** devem estar entre 50cc e 2500cc.
2. Caso qualquer uma dessas regras seja violada, lance a exceção **DadosVeiculoInvalidosException** detalhando o motivo no texto da mensagem.
3. Garanta que na classe **Main** a mensagem da exceção seja exibida de forma clara ao usuário no console.

Seção 2: Desafios de Integração com Banco de Dados (CRUD Completo)

Observação: No código da aula, implementamos as operações de **Create** e **Read**. Agora, complete o CRUD adicionando o **Update** e o **Delete**.

Desafio 1: Exclusão de Veículos (Delete)

1. Crie o método **public static boolean deletarVeiculo(int id) throws SQLException** na classe **ConexaoBanco**.
2. O método deve executar o comando SQL **DELETE FROM veiculos WHERE id = ?**.
3. Na classe **Main**, adicione a opção **"4. Excluir Veículo por ID"** no Menu Principal.
4. Teste informando um ID existente e verifique (via consulta no banco ou pela listagem da aplicação) se o registro foi removido com sucesso.

Desafio 2: Atualização de Registro (Update)

1. Crie o método **public static void atualizarAnoVeiculo(int id, int novoAno) throws SQLException, DadosVeiculoInvalidosException** na classe **ConexaoBanco**.
2. Aplique a validação de ano antes de disparar o comando **UPDATE veiculos SET ano = ? WHERE id = ?**.
3. Adicione a opção **"5. Atualizar Ano de um Veículo"** no Menu Principal.

Seção 3: Desafio Avançado (Programação Paralela / Threads)

Desafio 3: Thread de Alerta de Frota Antiga

1. Crie uma segunda classe que implemente `Runnable` chamada `AlertaFrotaAntigaThread`.
 - a. A thread deve rodar continuamente em segundo plano com intervalo de **20 segundos** (`Thread.sleep(20000)`).
 - b. A cada ciclo, ela deve consultar a lista de veículos cadastrados e verificar se existe algum veículo com ano de fabricação **anterior a 2010**.
 - c. Se houver, a thread deve imprimir no console um aviso formatado em destaque: **[ALERTA DE MANUTENÇÃO] Atenção: Existem X veículo(s) antigo(s) (ano < 2010) necessitando de revisão na frota!**
 - d. Registre e inicie a execução dessa thread no método **main** da classe **Main**.

Guia de Autoavaliação do Aluno

- a. ☐ O projeto compila sem erros no IntelliJ IDEA utilizando o comando `mvn clean compile`.
- b. ☐ Os novos tipos de veículos aparecem corretamente na tabela do banco SQLite (frota.db).
- c. ☐ As mensagens de exceção são informativas e não interrompem a execução do menu principal.

Gabarito e Soluções Esperadas**Respostas das Questões Objetivas:**

Questão	Resposta
1	c (RuntimeException)
2	c (PreparedStatement)
3	b (t.start())
4	b (Pode herdar de uma superclasse e implementar várias interfaces)
5	c (public static void main(String[] args))
6	b (Classes do mesmo pacote e subclasses em qualquer pacote)
7	c (<i>package-private</i> / apenas classes do mesmo pacote)
8	b (rs.next())
9	b (Atributos privados e métodos públicos/protegidos com regras de acesso)
10	b (Concorrência gerencia o progresso de tarefas; paralelismo as executa simultaneamente em hardware multi-core)

Resposta das Questões Subjetivas:**11. Exceções Customizadas:**

Resposta esperada: Exceções customizadas trazem clareza ao domínio (*ubiquitous language*), deixam o código autoexplicativo e permitem capturar e tratar erros de regras de negócio de forma centralizada e diferenciada das exceções de infraestrutura/sistema.

12. API JDBC:**Resposta esperada:**

```
String sql = "SELECT modelo, placa FROM veiculos WHERE placa = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, "ABC-1234");
    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            String modelo = rs.getString("modelo");
            System.out.println("Veículo encontrado: " + modelo);
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

13. Threads (run vs start):

Resposta esperada: Chamar `run()` diretamente apenas executa a lógica do método dentro da mesma thread corrente (thread principal / *main*), sem criar uma nova linha de execução na JVM. Para delegar a execução em paralelo a uma nova thread gerenciada pelo SO/JVM, deve-se invocar o método `.start()`.

14. Herança e Interfaces:**Resposta esperada:**

```
public interface Rastreavel {
    void atualizarPosicao(double lat, double lng);
}

public class Caminhao extends Veiculo implements Rastreavel {
    @Override
    public void atualizarPosicao(double lat, double lng) {
        // Lógica de atualização
    }
}
```

15. Encapsulamento:

Resposta esperada: O método `validarFraude` é um detalhe interno de implementação. Ele deve ser `private` para impedir que chamadores externos ignorem o fluxo principal de executar `Transacao` ou

executem a validação de forma descontextualizada. Se fosse public, outras classes poderiam acioná-lo isoladamente, violando o encapsulamento e a regra de negócio centralizada.